

Analysis of Algorithms (BDSF25M/A)

Date: 27-Apr-2026

Quiz 04

Time: 50 min

Roll No: BDSF24M029 Name: Abdullah Ayub

Grading	CLO-1	CLO-2	CLO-3	CLO-4
	/	/	/	/

Write brute-force, memorized and DP based algorithms for maximum sub-array sum problem, with there time and space complexities. Also, just mention, the running time for divide and conquer algorithm for the same problem. Think of its both 1D DP and 2D DP, plus greedy solution, and in case possible, comment about them.

Brute Force Algorithm:

```
brute-force-max-subarray(A)
1  maxSum = -∞
2  for i = 1 to n      — n
3      Sum = 0
4      for j = i to n  — n
5          Sum = Sum + A[j]
6          if Sum > maxSum
7              maxSum = Sum
8  return maxSum
```

Time complexity :

$O(n^2)$

Space complexity :

$O(1)$

Memorized (Top Down Dp) Solution :

Define: $memo[i]$ = maximum subarray sum ending at index i

Recurrence: $memo[i] = \max(A[i], memo[i-1] + A[i])$

Algorithm:

```
memorized-max-subarray(A)
  initialize memo[1...n] as undefined
  maxSum = -∞
  for i = 1 to n {
    maxSum = max(maxSum, Compute(i))
  }
  return maxSum
```

Compute(i)

if $memo[i]$ is defined
return $memo[i]$

if $i == 1$
 $memo[i] = A[i]$

else
 $memo[i] = \max(A[i], \text{Compute}(i-1) + A[i])$

Space Complexity: $O(n)$

Time Complexity: $O(n)$

Dynamic programming:

Recurrence relation:

$$dp[i] = \max(A[i], dp[i-1] + A[i])$$

where $dp[i]$ = maximum subarray sum ending at i

algorithm:

dp-max-subarray(A)

$$dp[1] = A[1]$$

$$\text{max sum} = dp[1]$$

for $i = 2$ to n

$$dp[i] = \max(A[i], dp[i-1] + A[i])$$

if $dp[i] > \text{max sum}$

$$\text{max sum} = dp[i]$$

return max sum.

Time complexity: $O(n)$

Space complexity: $O(n)$

Divide & Conquer running time:

$$T(n) = 2T(n/2) + O(n)$$

using master theorem: $T(n) = O(n \log n)$

1D DP: formula used: $\text{current} = \max(A[i], \text{current} + A[i])$

Time: $O(n)$, Space: $O(1)$

2D DP: $dp[i][j] = dp[i][j-1] + A[j]$

↳ sum of subarray from i to j

Time: $O(n^2)$, Space: $O(n^2)$

Greedy Algo: 1D dp algorithm is also considered as a greedy algorithm because

: if current sum becomes negative

starting a new subarray is always better